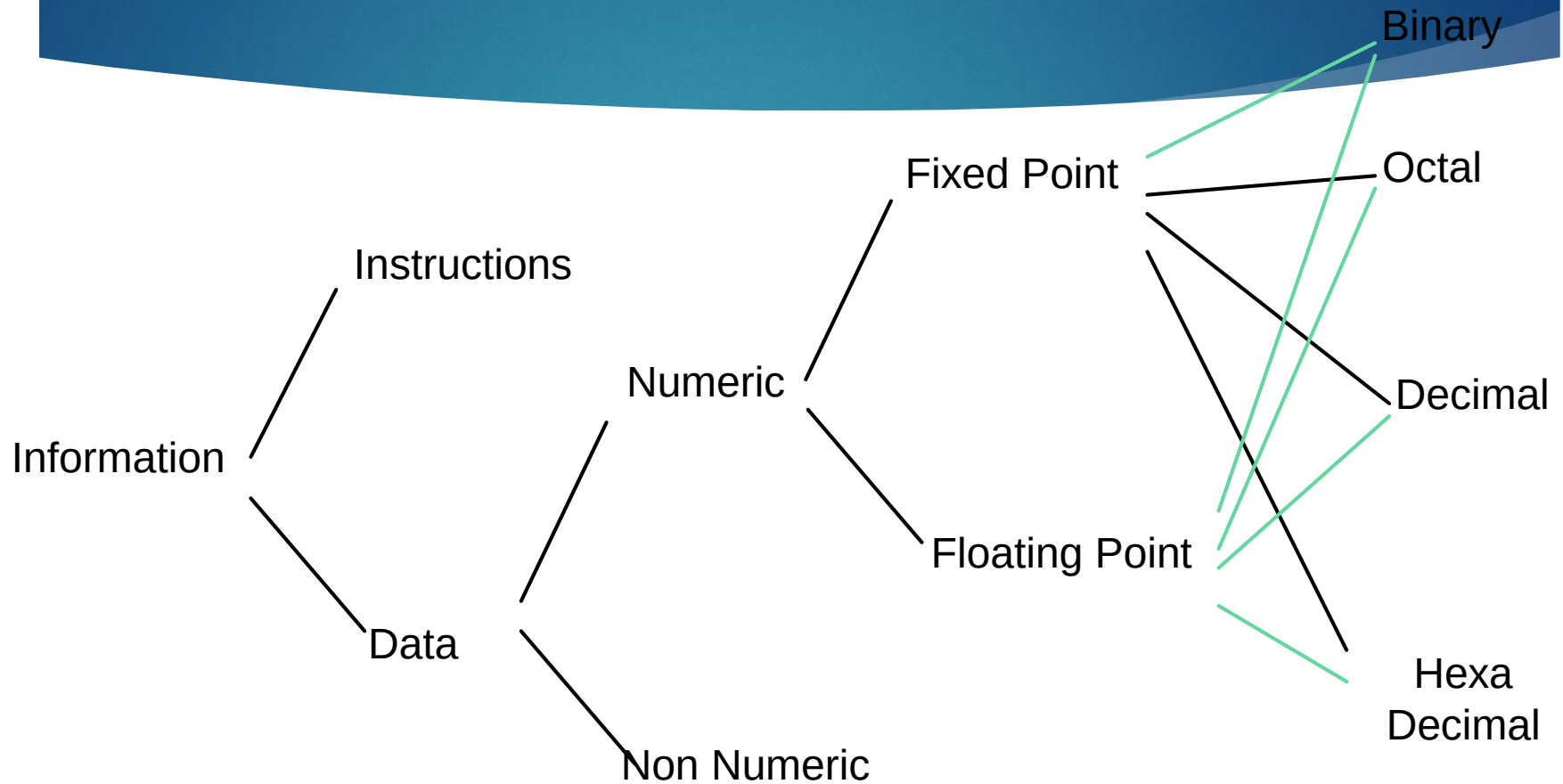




Data Representation and Computer Arithmetic

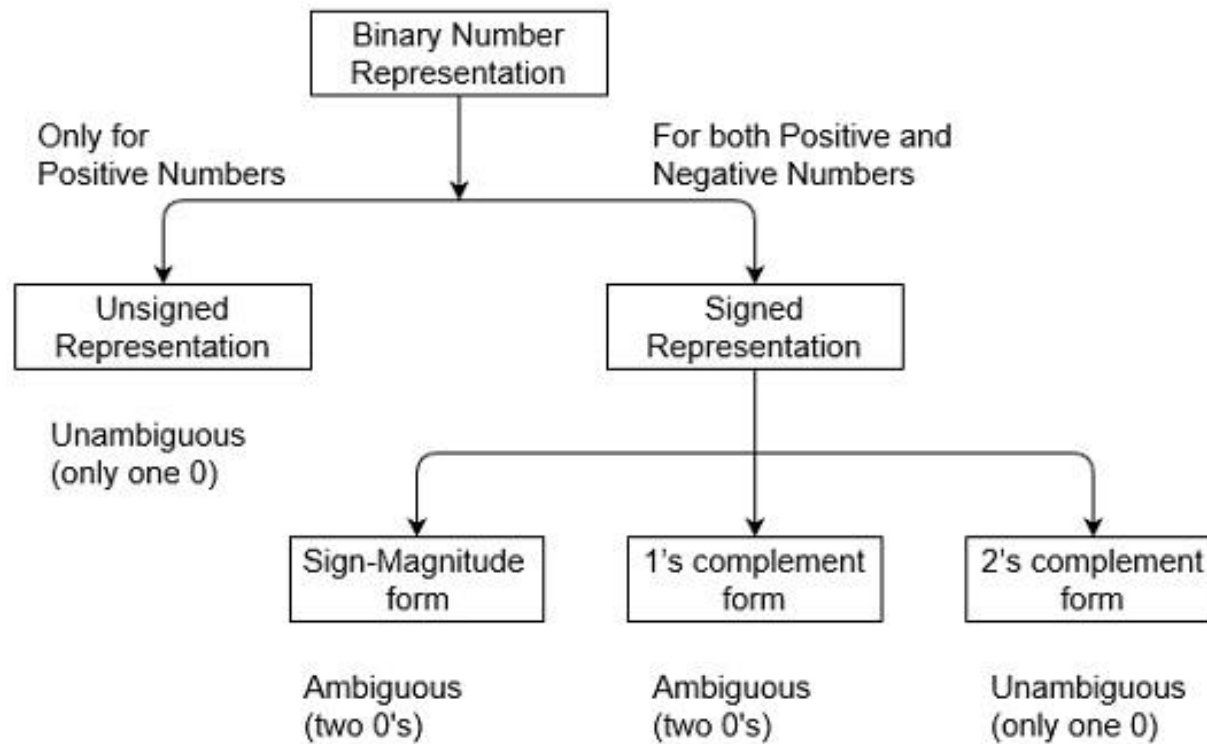
ANISHA M. LAL

Data Representation



Integer /Fixed point representations

- ▶ There are 4 commonly known (1 uncommon) integer representations.
- ▶ All have been used at various times for various reasons.
 - ▶ unsigned
 - ▶ sign magnitude
 - ▶ one's complement
 - ▶ two's complement
 - ▶ biased (not commonly known)



Unsigned Representation

- ▶ In general, if an n-bit sequence of binary digits $a_{n-1}a_{n-2} \dots a_1a_0$ is interpreted as an unsigned integer A, its value is

$$A = \sum_{i=0}^{n-1} 2^i a_i$$

- ▶ Range: 0 to $2^n - 1$
- ▶ For n bits.
- ▶ Ex: 0 to 15 for 4 bits
($n=4$, $2^4 - 1$ is 15)

Binary	decimal	hex	binary	decimal	hex
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	10	a
0011	3	3	1011	11	b
0100	4	4	1100	12	c
0101	5	5	1101	13	d
0110	6	6	1110	14	e
0111	7	7	1111	15	f

Sign magnitude Representation

- ▶ The simplest form of representation that employs a sign bit is the sign-magnitude representation.
- ▶ In an n-bit word, the rightmost bits hold the magnitude of the integer.
- ▶ The most significant (leftmost) bit in the word as a sign bit.
- ▶ Sign bit = 0 => positive
- ▶ Sign bit = 1 => negative.
- ▶ Ex: + 18 = 00010010; - 18 = 10010010
- ▶ Range: $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$
- ▶ To get the additive inverse of a number, just flip (not, invert, complement, negate) the sign bit.

Sign magnitude representation

- ▶ The general case can be expressed as follows:

$$A = \begin{cases} \sum_{i=0}^{n-2} 2^i a_i & \text{if } a_{n-1} = 0 \\ -\sum_{i=0}^{n-2} 2^i a_i & \text{if } a_{n-1} = 1 \end{cases}$$

- ▶ Drawbacks
 - ▶ Addition and subtraction require a consideration of both the signs of the numbers and their relative magnitudes to carry out the required operation.
 - ▶ There are two representations of 0

1's Complement Representation

- ▶ Historically important, and we use this representation to get 2's complement integers
- ▶ In the past, early computers built by Semour Cray (while at CDC) were based on 1's comp. integers.
- ▶ Positive integers use the same representation as unsigned; 00000 is 0; 00111 is 7;
- ▶ Negation (finding an additive inverse) is done by taking a bit wise complement of the positive representation.
- ▶ EXAMPLES: 11100 (This must be a negative number)
- ▶ 00011 is +3, so 11100 must be -3
- ▶ Things to notice:
 - ▶ Any negative number will have a 1 in the MSB.
 - ▶ There are 2 representations for 0; 00000 and 11111.
 - ▶ Range is : $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$

2's Complement

- ▶ The most significant bit represents a sign of the number
- ▶ **Range:** -2^{n-1} through $2^{n-1} - 1$
- ▶ **Number of Representations of Zero:** One
- ▶ **Negation:** Take the Boolean complement of each bit of the corresponding positive number, then add 1 to the resulting bit pattern viewed as an unsigned integer.
- ▶ **Expansion of Bit Length:** Add additional bit positions to the left and fill in with the value of the original sign bit.
- ▶ **Overflow Rule:** If two numbers with the same sign (both positive or both negative) are added, then overflow occurs if and only if the result has the opposite sign.

Pros and cons of integer representation

- ▶ Signed magnitude representation:
 - ▶ 2 representations for 0
 - ▶ Simple
 - ▶ Need to consider both sign and magnitude in arithmetic
 - ▶ Different logic for addition and subtraction
- ▶ 1's complement representation:
 - ▶ 2 representations for 0
 - ▶ Complexity in performing addition and subtraction
- ▶ 2's complement representation:
 - ▶ Only one representation for 0
 - ▶ Arithmetic works easily
 - ▶ Negation is fairly easily

Comparing the signed number systems

- Here are all the 4-bit numbers in the different systems.
- *Positive numbers are the same in all three representations.*
- Signed magnitude and one's complement have *two* ways of representing 0. This makes things more complicated.
- Two's complement has asymmetric ranges; there is one more negative number than positive number. Here, you can represent -8 but not +8.
- However, two's complement is preferred because it has only one 0, and its addition algorithm is the simplest.

Decimal	S.M.	1's comp.	2's comp.
7	0111	0111	0111
6	0110	0110	0110
5	0101	0101	0101
4	0100	0100	0100
3	0011	0011	0011
2	0010	0010	0010
1	0001	0001	0001
0	0000	0000	0000
-0	1000	1111	—
-1	1001	1110	1111
-2	1010	1101	1110
-3	1011	1100	1101
-4	1100	1011	1100
-5	1101	1010	1011
-6	1110	1001	1010
-7	1111	1000	1001
-8	—	—	1000

Ranges of the signed number systems

- How many negative and positive numbers can be represented in each of the different systems on the previous page?

	Unsigned	Signed Magnitude	One's complement	Two's complement
Smallest	0000 (0)	1111 (-7)	1000 (-7)	1000 (-8)
Largest	1111 (15)	0111 (+7)	0111 (+7)	0111 (+7)

- In general, with n-bit numbers including the sign, the ranges are:

	Unsigned	Signed Magnitude	One's complement	Two's complement
Smallest	0	$-(2^{n-1}-1)$	$-(2^{n-1}-1)$	-2^{n-1}
Largest	2^n-1	$+(2^{n-1}-1)$	$+(2^{n-1}-1)$	$+(2^{n-1}-1)$